머신러닝 앙상블

01

앙상블 이해

앙상블(Ensemble)

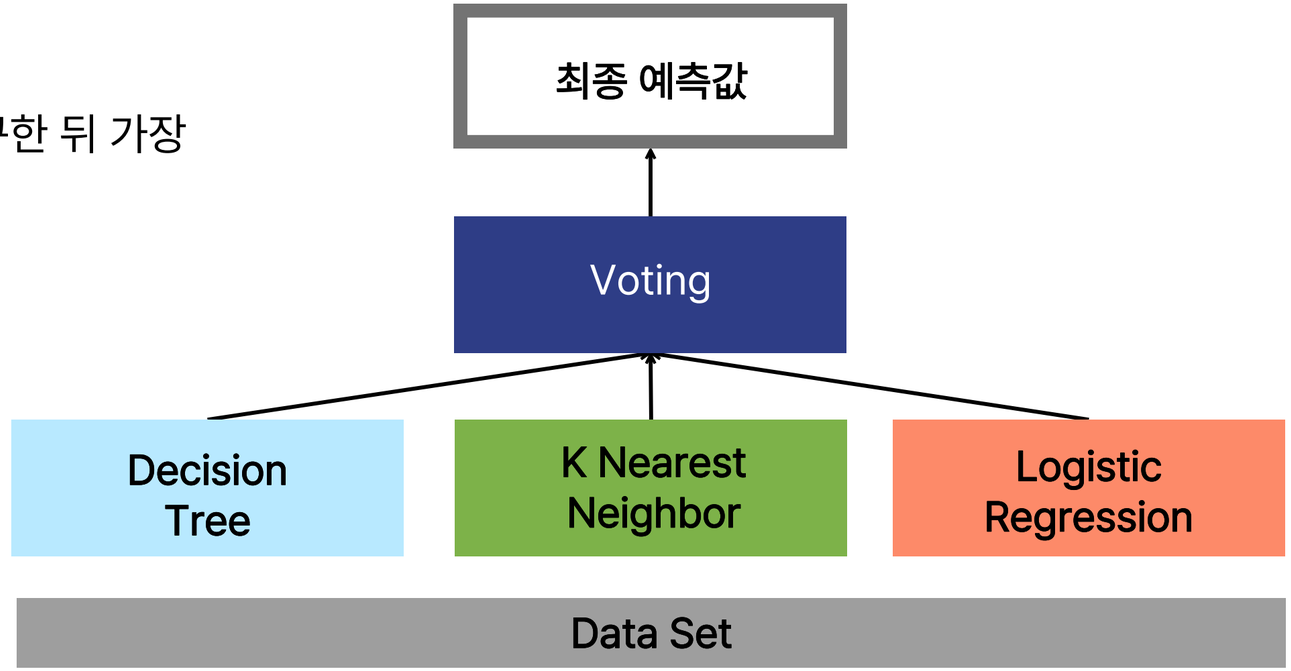
- 여러 개의 머신러닝 모델을 결합하여 예측 성능을 향상시키는 기법
  - 배깅, 부스팅, 스태킹과 같은 다양한 방법을 통해 모델의 강점은 극대화하고 약점은 보완할 수 있음
- 캐글(Kaggle)과 같은 많은 기계학습 경쟁에서 상위 순위를 차지하고 있음

02

보팅(Voting)

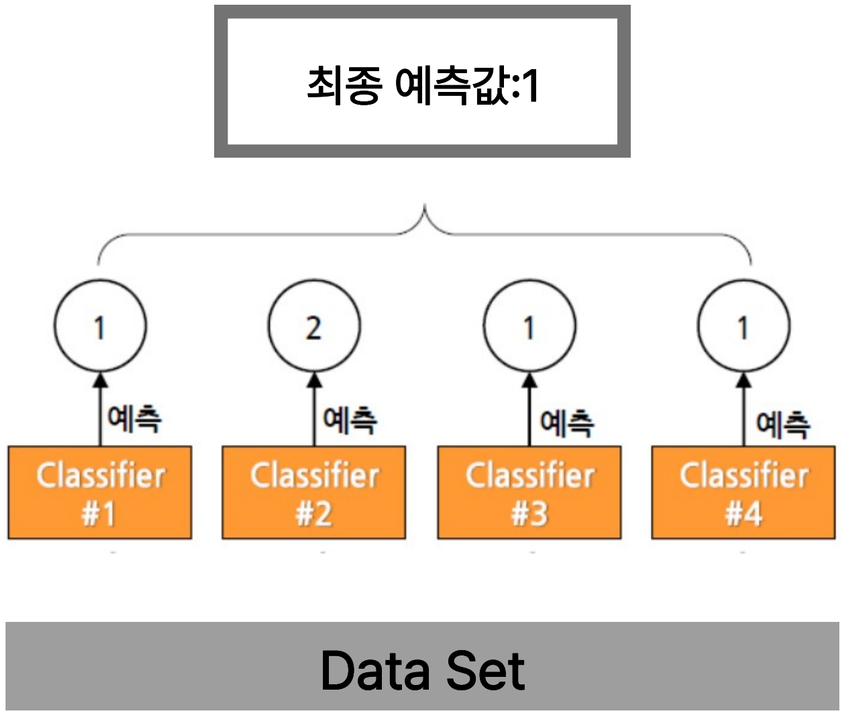
보팅(Voting)

- 여러개의 분류기가 투표를 통해 최종 예측 결과를 결정하는 방식
- 다른 유형의 알고리즘 기반 분류기 사용
- 하드 보팅 (Hard Voting)
  - 다수결 분류기가 예측한 값이 최종 결괏값
- 소프트 보팅 (Soft Voting)
  - 모든 분류기가 예측한 레이블 값의 결정 확률 평균을 구한 뒤 가장 확률이 높은 레이블 값을 최종 결과로 선정

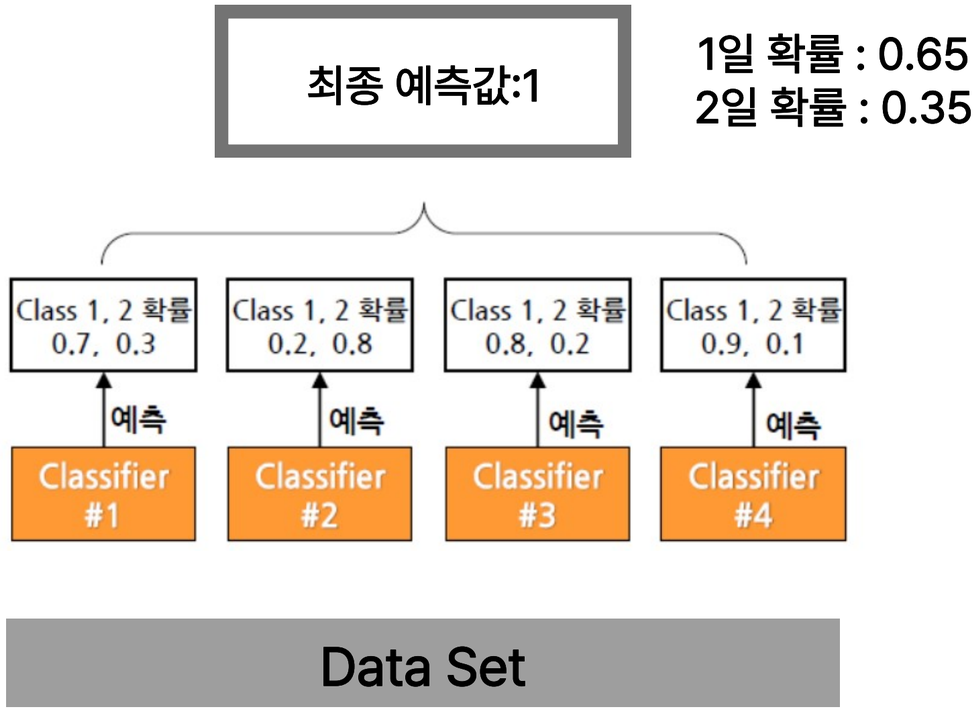


하드 보팅과 소프트 보팅

하드보팅



소프트 보팅



보팅 모델링 예

```python
# 1. 모듈 불러오기
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import *
```

✓ 0.0s

```python
# 2. 모델 설정
estimators = [('dt', DecisionTreeClassifier()),
              ('knn', make_pipeline(StandardScaler(), KNeighborsClassifier())),
              ('lr', LogisticRegression(max_iter=1000)),
              ('xgb', XGBClassifier()),
              ('lgbm', LGBMClassifier())]
model = VotingClassifier(estimators=estimators)
```

✓ 0.0s

```python
# 3. 학습하기
model.fit(X_train, y_train)
```

- 여러 모델을 기반으로 한다고 해서 무조건 보팅 모델 성능이 좋은 것은 아님
- 데이터 특성과 분포 등 여러 요건에 따라 기반 모델 성능이 더 좋을 수도 있음

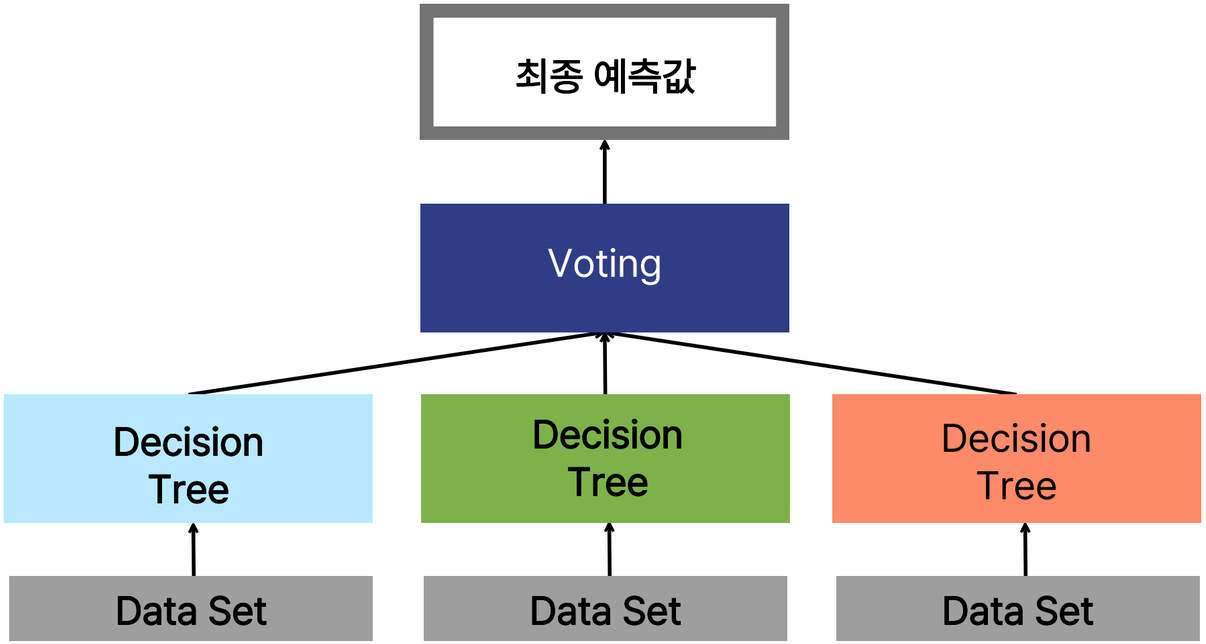
03

배깅(Bagging)

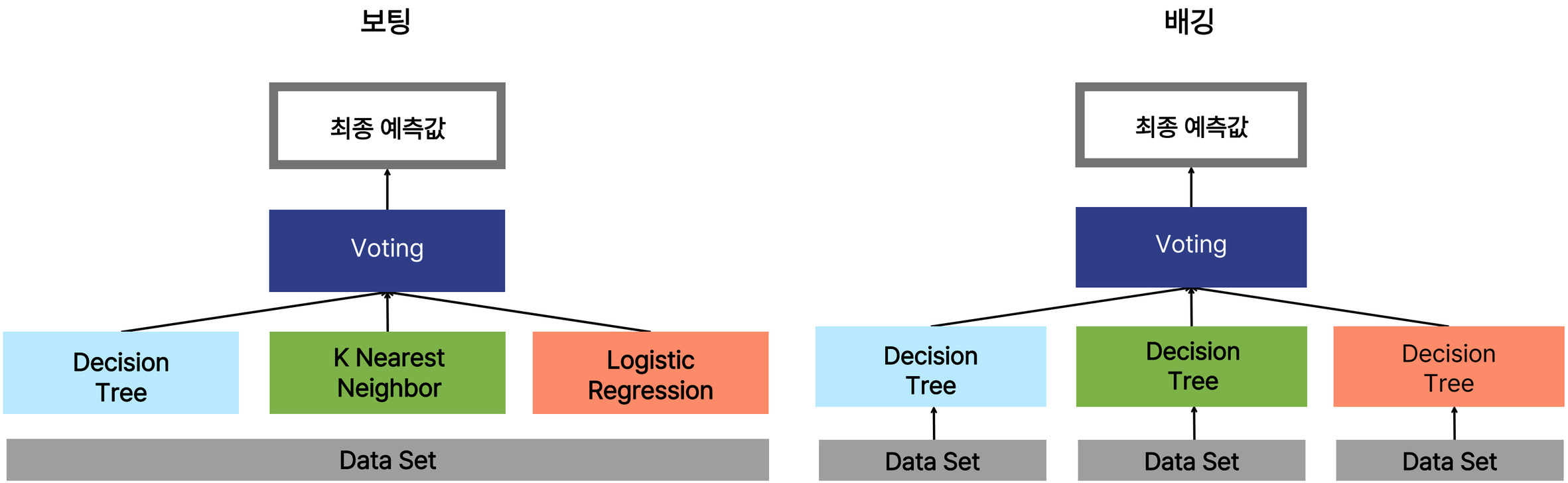
배깅(Bagging)

- Boostrap Aggregating 의 약자
- 데이터로부터 부트스트랩한 데이터로 각자 학습시킨 후, 학습된 모델의 결과를 집계하여 최종 결과를 얻는 방법

  (부트스트랩 샘플 : 원본 데이터에서 복원추출을 해서 만든 여러개의 샘플 데이터)
- 같은 유형의 알고리즘 기반 분류기 사용
- 데이터 분할시 중복을 허용(복원 추출)
- 분류 문제는 투표방식으로 결과를 집계
- 회귀 문제는 평균으로 결과를 집계

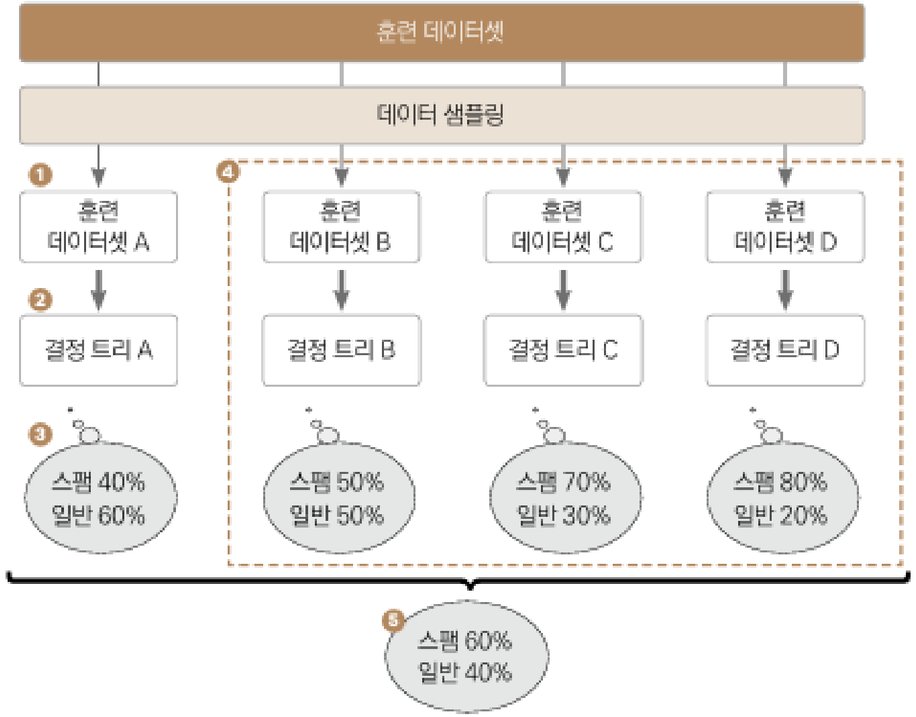


보팅과 배깅 비교


보팅
최종 예측값
Voting
Decision Tree
K Nearest Neighbor
Logistic Regression
Data Set
배깅
최종 예측값
Voting
Decision Tree
Decision Tree
Decision Tree
Data Set
Data Set
Data Set

랜덤 포레스트(Random Forest)

- 배깅(Bagging)의 가장 대표적인 알고리즘
- 여러 개의 의사결정나무(Decision Tree) 를 조합하여 예측 성능을 향상시키는 알고리즘
- 모델들이 개별적으로 학습을 수행한 뒤 모든 결과를 집계하여 최종 결과 결정
- 랜덤포레스트는 부트스트랩 샘플링과 함께 각 분할에서 일부 특성만 랜덤하게 사용해 트리 간 다양성을 높임



source: [Must Have]머신러닝,딥러닝 문제해결 전략

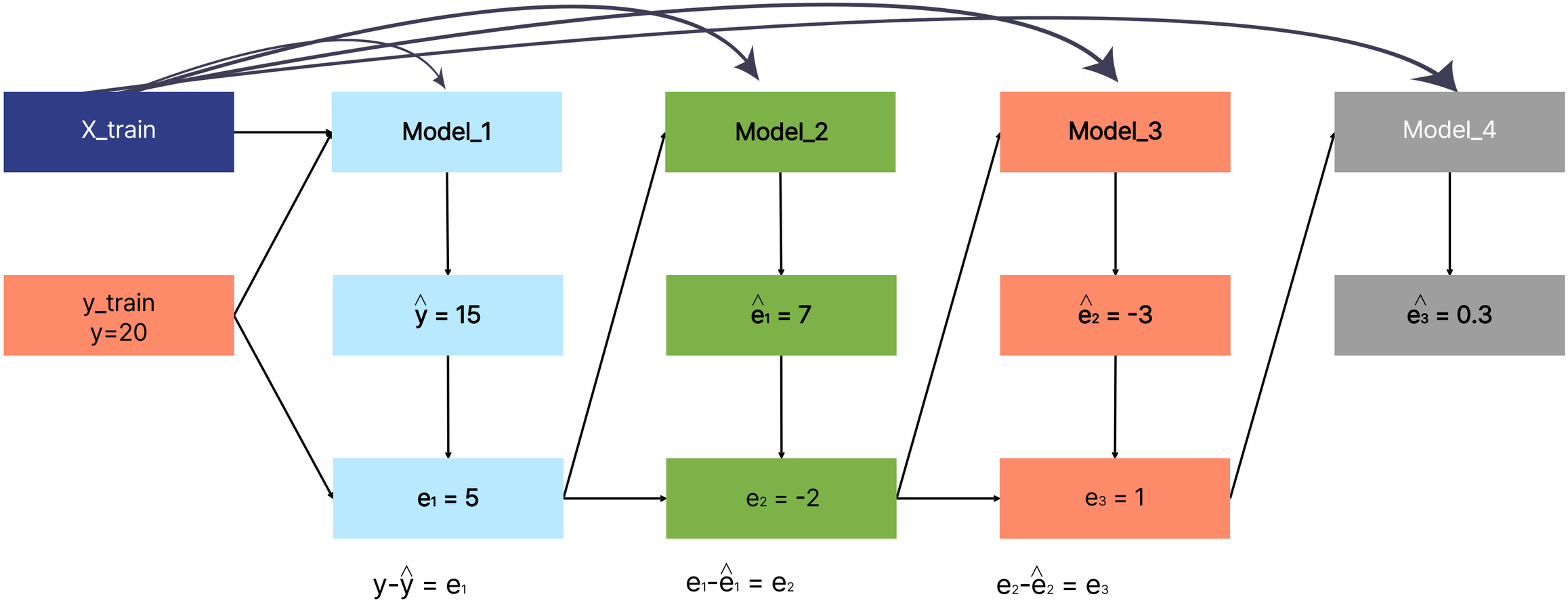
04

부스팅(Boosting)

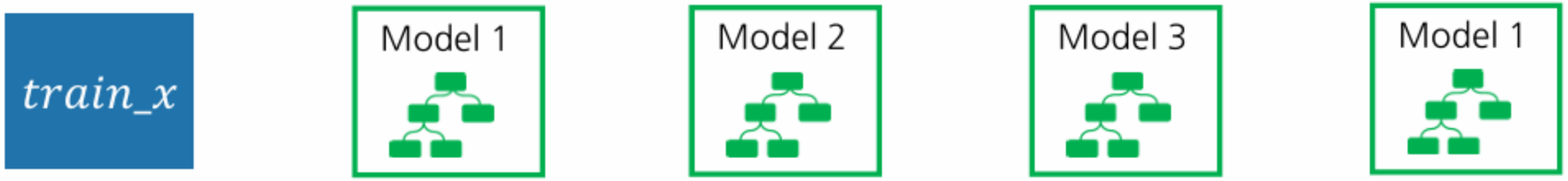
부스팅(Boosting)

- 같은 유형의 알고리즘 기반 분류기 여러개를 순차적으로 학습
- 이전 분류기 예측이 틀린 데이터에 대해 올바르게 예측할 수 있도록 다음 분류기에게 가중치(Weight)를 부여하면서 학습과 예측을 진행
- 계속 가중치를 부스팅하며 학습을 진행하는 방식
- 예측 성능이 뛰어나 앙상블 학습 중에 많이 쓰이는 방식
- 배깅에 비해 성능이 좋지만 속도가 느리고 과적합 발생 가능성이 있으므로 상황에 맞게 적절히 사용해야 됨
- 배깅과 부스팅 비교
  - 배깅은 분류기들이 학습 시 상호 영향을 주지 않고 학습이 끝난 후 그 결과를 종합하는 방법
  - 부스팅은 이전 분류기 학습 결과를 토대로 다음 분류기 학습 데이터 샘플 가중치를 조정해 학습을 진행하는 방법
- 대표적인 부스팅 방법 : XGBoost, LightGBM

부스팅 원리


X_train
y_train
y=20
Model_1
$\hat{y}$ = 15
e_1 = 5
Model_2
$\hat{e}_1$ = 7
e_2 = -2
Model_3
$\hat{e}_2$ = -3
e_3 = 1
Model_4
$\hat{e}_3$ = 0.3
$y-\hat{y} = e_1$
$e_1-\hat{e}_1 = e_2$
$e_2-\hat{e}_2 = e_3$

부스팅 원리



$$y = m_1(x) + err$$

$$y = m_1(x) + err_1(x)$$

$$y = m_1(x) + \boxed{m_2(x) + err_2(x)}$$

$$y = m_1(x) + m_2(x) + \boxed{m_3(x) + err_3(x)}$$

$$y = m_1(x) + m_2(x) + m_3(x) + \boxed{m_4(x) + err_4(x)}$$

$$err_1(x) = \boxed{m_2(x) + err_2(x)}$$

$$err_2(x) = \boxed{m_3(x) + err_3(x)}$$

$$err_3(x) = \boxed{m_4(x) + err_4(x)}$$

XGBoost(eXtremeGradient Boosting)

- 부스팅을 구현한 대표적인 알고리즘 중 하나가 GBM(Gradient Boost Machine)
- GBM 알고리즘을 병렬 학습이 가능하도록 구현한 것이 XGBoost
- 회귀, 분류 문제를 모두 지원하며, 성능과 자원 효율이 좋아 많이 사용됨
- XGBoost 장점

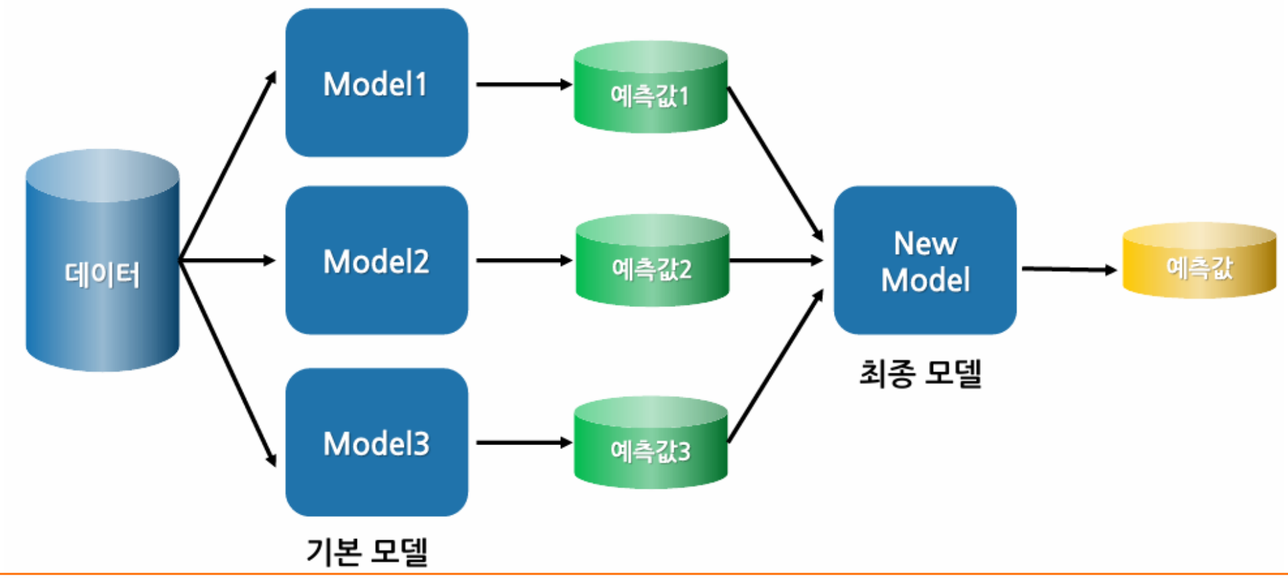
장점	설명
높은 예측 성능	• 분류와 회귀 모두에서 뛰어난 성능을 보임
빠른 수행 시간(GBM 대비)	• 병렬 수행 및 다양한 기능으로 GBM에 비해 빠르게 수행됨 • 하지만 여전히 다른 알고리즘에 비해서는 느림
가지치기(Tree Pruning)	• max_depth 등의 하이퍼파라미터로 가지치기를 할 수 있음 • Tree Pruning 기능으로 성능에 이점이 없는 분할은 가지치기 할 수 있음
규제(Regularization)	• GBM에는 없었던 과적합 규제 기능을 가지고 있음
내장된 교차 검증 (Cross Validation)	• 반복 수행시마다 내부적으로 학습 데이터와 검증 데이터에 대한 교차 검증을 수행 • 지정된 반복 횟수가 아닌 교차 검증을 통해 성능을 확인하여 필요시 조기 중단 가능
결측치 자체 처리	• use_missing=True 옵션을 지정하면 결측치를 모델 내부에서 처리(LightGBM의 경우)

05

스태킹(Stacking)

스태킹(Stacking)

- 여러 모델의 예측값을 최종 모델의 학습 데이터로 사용하여 예측하는 방법
  - 예) KNN, Logistic Regression, XGBoost 모델을 사용하여 예측값을 구한 후
    다른 모델인 최종 모델인 LGBM 모델의 학습 데이터로 사용
- 실무에서는 성능 향상이 확실하고 운영 복잡도를 감당할 수 있을 때 사용
- 캐글(Kaggle)과 같은 대회에서 성능을 조금이라도 끌어올리고 싶을 때 사용
- 서로 다른 강점을 가진 다양한 모델을 조합할수록 효과를 기대할 수 있음(기본 모델 4개 이상 선택해야 좋은 결과 기대)



스태킹(Stacking)

- Base Models (기본 모델) 선정
  - 서로 다른 특성과 강점을 가진 모델 선택
    - 랜덤포레스트(규모 데이터에 강함)
    - XGBoost(복잡한 상호작용 처리)
- Meta Model (메타 모델) 선정
  - Base Models의 출력 결과를 입력으로 받아 최종 예측
  - 메타 모델로는 선형 회귀, 로지스틱 회귀, 또는 간단한 부스팅 모델 사용
- 학습 방법
  - 데이터를 두 부분으로 나눔
    - Fold 1: Base Models 학습용
    - Fold 2: Base Models의 예측값을 사용해 Meta Model 학습
- Cross-Validation 사용
  - 교차 검증을 통해 Base Models와 Meta Model 모두 일반화된 성능 확보

스태킹 모델링 예제 코드

```python
# 1. 모듈 불러오기
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import StackingClassifier
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import *
```

✓ 0.0s

```python
# 2. 모델 설정
estimators = [('dt', DecisionTreeClassifier()),
              ('knn', make_pipeline(StandardScaler(), KNeighborsClassifier())),
              ('lr', LogisticRegression(max_iter=1000)),
              ('xgb', XGBClassifier())]
model = StackingClassifier(estimators=estimators,
                           final_estimator=LGBMClassifier())
```

✓ 0.0s

```python
# 3. 학습하기
model.fit(X_train, y_train)
```

모델종류

모델명	내용	장단점	기법
랜덤포레스트	- 여러 개의 결정 트리를 랜덤하게 학습한 후, 이들의 결과를 평균(회귀) 또는 투표(분류)로 결합	- 과적합 방지 - 차원 데이터에 강함 - 많은 트리를 학습	배깅
GBM	- Gradient Boosting 알고리즘을 기반으로, 이전 트리의 오류를 수정하며 새로운 트리를 추가하여 모델을 학습	- 과적합 방지를 위해 규제 옵션 제공 - 느린 학습 속도	부스팅
XGBoost	- Gradient Boosting에 기반하며, 각 트리가 이전 트리의 오류를 줄이는 방식으로 학습	- 높은 정확도 - 과적합 방지 기법 내장 (e.g., early stopping, 규제) - GPU 지원	부스팅
LightGBM	- Gradient Boosting을 기반으로 하며, 리프 중심 트리 분할 방식을 사용하여 더 깊은 트리를 생성	- 대규모 데이터셋에서 빠른 학습 - 작은 데이터셋에서는 과적합 가능	부스팅
CatBoost	- Gradient Boosting 기반으로, 카테고리형 변수를 자동 처리	- 카테고리형 데이터 처리에 최적화 - GPU 및 분산 학습 지원 - 학습 시간이 비교적 오래 걸릴 수 있음	부스팅